

CO3 AT1 – Experiments

CSA 1419 – Compiler Design

Name: K. Madhulika

Reg no: 192524190

Q1.

You are designing a compiler module that evaluates arithmetic expressions using syntax-directed definitions (SDD). The system must:

- (i) Accept arithmetic expressions with operators +, -, *, / and parentheses
- (ii) Construct a syntax tree and evaluate expressions using synthesized attributes
- (iii) Respect operator precedence and associativity rules
- (iv) Produce the final computed value of the expression

Demonstrate how SDD rules are applied for evaluating expressions and explain the bottom-up

evaluation process using annotated parse trees.

Program:

class Node:

```
def __init__(self, value, left=None, right=None):  
    self.value = value  
    self.left = left  
    self.right = right  
    self.val = None
```

class Parser:

```
def __init__(self, expression):  
    self.expression = expression  
    self.pos = 0  
  
def skip_spaces(self):  
    while self.pos < len(self.expression) and self.expression[self.pos].isspace():  
        self.pos += 1  
  
# E -> E + T | E - T | T
```

```

def expression_rule(self):
    node = self.term()
    while True:
        self.skip_spaces()
        if self.pos < len(self.expression) and self.expression[self.pos] == '+':
            self.pos += 1
            right = self.term()
            node = Node('+', node, right)
        elif self.pos < len(self.expression) and self.expression[self.pos] == '-':
            self.pos += 1
            right = self.term()
            node = Node('-', node, right)
        else:
            break
    return node

# T -> T * F | T / F | F

def term(self):
    node = self.factor()
    while True:
        self.skip_spaces()
        if self.pos < len(self.expression) and self.expression[self.pos] == '*':
            self.pos += 1
            right = self.factor()
            node = Node('*', node, right)
        elif self.pos < len(self.expression) and self.expression[self.pos] == '/':
            self.pos += 1
            right = self.factor()
            node = Node('/', node, right)
        else:

```

```

        break

    return node

# F -> (E) | num
def factor(self):

    self.skip_spaces()

    # Parentheses
    if self.pos < len(self.expression) and self.expression[self.pos] == '(':

        self.pos += 1

        node = self.expression_rule()

        self.skip_spaces()

        if self.pos < len(self.expression) and self.expression[self.pos] == ')':

            self.pos += 1

        else:

            raise ValueError("Missing closing parenthesis")

        return node

    # Number
    else:

        start = self.pos

        while self.pos < len(self.expression) and (

            self.expression[self.pos].isdigit()

            or self.expression[self.pos] == '.'

        ):

            self.pos += 1

        if start == self.pos:

            raise ValueError("Invalid expression")

        number = float(self.expression[start:self.pos])

        node = Node(number)

        # Synthesized attribute
        node.val = number

```

```

        return node

# Bottom-up evaluation
def evaluate(node):

    # Leaf node
    if node.left is None and node.right is None:

        return node.val

    # Evaluate left child
    left_value = evaluate(node.left)

    # Evaluate right child
    right_value = evaluate(node.right)

    # Apply SDD semantic rules
    if node.value == '+':

        node.val = left_value + right_value
    elif node.value == '-':

        node.val = left_value - right_value
    elif node.value == '*':

        node.val = left_value * right_value
    elif node.value == '/':

        if right_value == 0:

            raise ZeroDivisionError("Cannot divide by zero")

        node.val = left_value / right_value

    return node.val

# Print syntax tree with synthesized values
def print_tree(node, prefix="", is_left=True):

    if node is None:

        return

    # Print current node

    if isinstance(node.value, float):

        text = str(node.value)

```

```

else:
    text = str(node.value)
    print(prefix + (" |—— " if is_left else " |—— ")
          + text + " [val=" + str(node.val) + "]")
    # Print children
    if node.left is not None:
        new_prefix = prefix + (" | " if is_left else " ")
        print_tree(node.left, new_prefix, True)

    if node.right is not None:
        new_prefix = prefix + (" | " if is_left else " ")
        print_tree(node.right, new_prefix, False)
print(" Syntax Directed Definition (SDD)")
print(" Arithmetic Expression Evaluation")
expression = input("\nEnter arithmetic expression: ")
try:
    # Create parser
    parser = Parser(expression)
    # Construct syntax tree
    tree = parser.expression_rule()
    # Check if complete expression was processed
    parser.skip_spaces()
    if parser.pos != len(expression):
        raise ValueError("Invalid expression")
    # Evaluate tree bottom-up
    result = evaluate(tree)
    # Display syntax tree
    print("\nSyntax Tree with Synthesized Attributes:")
    print_tree(tree, "", False)

```

```

# Display final result

print("\n")

print("Expression :", expression)

print("Final Result:", result)

print("")

except Exception as e:

    print("\nError:", e)

```

Output:

```

IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ADMIN/OneDrive/Documents/CO3 AT1 EXP-1 CD.py =====
=====
Syntax Directed Definition (SDD)
Arithmetic Expression Evaluation
=====

Enter arithmetic expression: 3+5*(2-1)

Syntax Tree with Synthesized Attributes:
├─ + [val=8.0]
│   └─ 3.0 [val=3.0]
│       └─ * [val=5.0]
│           └─ 5.0 [val=5.0]
│               └─ - [val=1.0]
│                   └─ 2.0 [val=2.0]
│                       └─ 1.0 [val=1.0]
=====

Expression : 3+5*(2-1)
Final Result: 8.0
=====
>>>

```

Q2.

In a compiler, intermediate code generation often uses postfix expressions for efficient evaluation. The system must:

- (i) Accept a postfix expression as input
- (ii) Use a stack-based approach to evaluate the expression

(iii) Demonstrate bottom-up computation of intermediate results

(iv) Display the final evaluated result

Write a C program to evaluate postfix expressions and explain how this process relates to S-attributed definitions in compilers.

Program:

```
#include <stdio.h>

#include <ctype.h>

#define MAX 100

int stack[MAX];

int top = -1;

/* Push an element into stack */

void push(int value)
{
    if (top == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }
    top++;
    stack[top] = value;
}

/* Pop an element from stack */

int pop()
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
        return -1;
    }
}
```

```

    return stack[top--];
}

/* Main function */
int main()
{
    char postfix[MAX];
    int i;
    int operand1, operand2, result;
    printf("Enter postfix expression: ");
    scanf("%s", postfix);
    for (i = 0; postfix[i] != '\0'; i++)
    {
        /* If character is an operand */
        if (isdigit(postfix[i]))
        {
            push(postfix[i] - '0');

            printf("Read %c -> Push %d\n",
                postfix[i], postfix[i] - '0');
        }
        /* If character is an operator */
        else
        {
            operand2 = pop();
            operand1 = pop();
            switch (postfix[i])
            {
                case '+':
                    result = operand1 + operand2;

```



```

        break;
    case '-':
        result = operand1 - operand2;
        break;
    case '*':
        result = operand1 * operand2;
        break;
    case '/':
        if (operand2 == 0)
        {
            printf("Error: Division by zero\n");
            return 1;
        }
        result = operand1 / operand2;
        break;
    default:
        printf("Invalid operator\n");
        return 1;
}

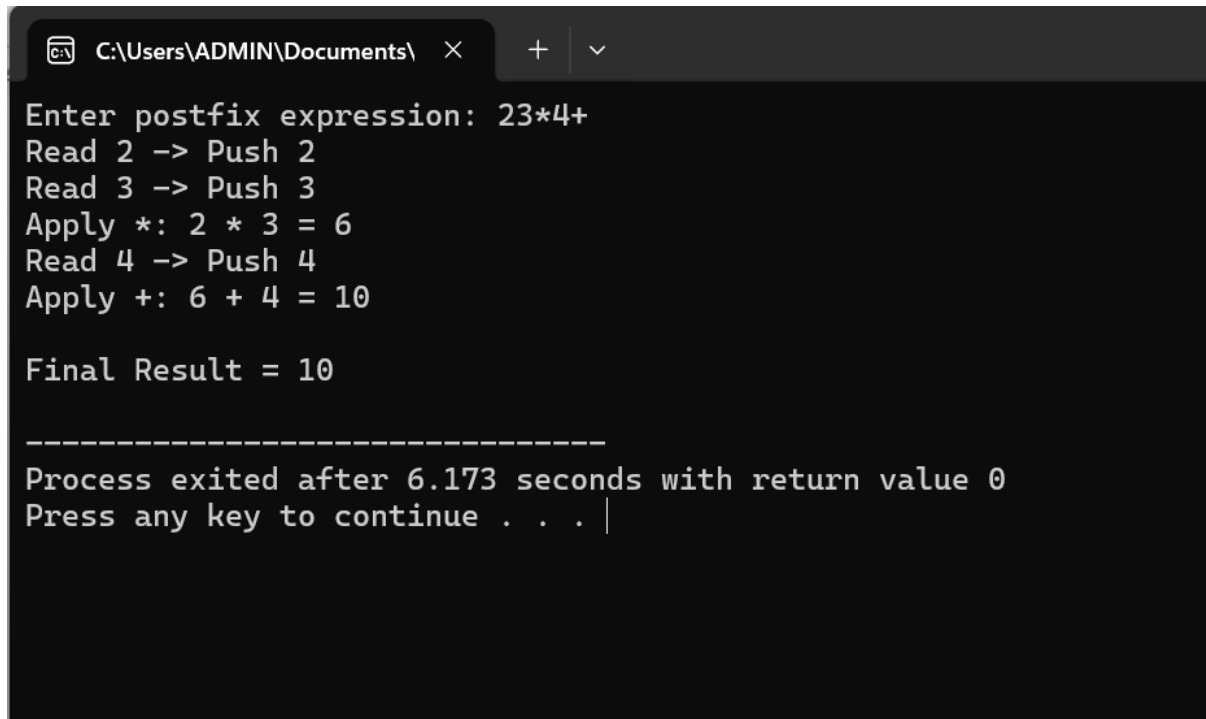
printf("Apply %c: %d %c %d = %d\n",
        postfix[i],
        operand1,
        postfix[i],
        operand2,
        result);
push(result);
}
}

result = pop();

```

```
printf("\nFinal Result = %d\n", result);  
  
return 0;  
  
}
```

Output:



```
C:\Users\ADMIN\Documents\ >  
Enter postfix expression: 23*4+  
Read 2 -> Push 2  
Read 3 -> Push 3  
Apply *: 2 * 3 = 6  
Read 4 -> Push 4  
Apply +: 6 + 4 = 10  
  
Final Result = 10  
  
-----  
Process exited after 6.173 seconds with return value 0  
Press any key to continue . . . |
```

Q3.

You are developing a compiler component that uses L-attributed definitions for expression evaluation. The system must:

- (i) Accept arithmetic expressions such as $a + b * c$
- (ii) Use inherited attributes to pass information during parsing
- (iii) Ensure correct operator precedence
- (iv) Perform evaluation using a top-down approach

Write a C program that simulates L-attributed evaluation and explain how inherited attributes are used to propagate values during parsing.

Program:

```
#include <stdio.h>
```

```
#include <ctype.h>
#include <string.h>
char expression[100];
int pos = 0;
double a, b, c;
/*
```

L-ATtributed Evaluation

Grammar:

$E \rightarrow T E'$

$E' \rightarrow + T E'$

$E' \rightarrow - T E'$

$E' \rightarrow \text{epsilon}$

$T \rightarrow F T'$

$T' \rightarrow * F T'$

$T' \rightarrow / F T'$

$T' \rightarrow \text{epsilon}$

$F \rightarrow \text{id}$

$F \rightarrow (E)$

The inherited attribute is represented by
the parameter 'inh'.

The value calculated so far is passed from
parent to child.

```
*/
/* Function declarations */
double E();
double EPrime(double inh);
double T();
double TPrime(double inh);
double F();
```

```

/* Skip spaces */
void skipSpaces()
{
    while (expression[pos] == ' ')
    {
        pos++;
    }
}

/* Get value of variable */
double getValue(char variable)
{
    if (variable == 'a')
        return a;
    if (variable == 'b')
        return b;
    if (variable == 'c')
        return c;
    return 0;
}

/*E -> T E*/
double E()
{
    double tValue;
    double result;
    printf("\nEntering E (Expression)\n");
    tValue = T();
    printf("T produced value = %.2f\n", tValue);
    /* Pass T's value as inherited attribute to E*/
    printf("Inherited value passed to E' = %.2f\n", tValue);
}

```

```

    result = EPrime(tValue);
    printf("E returns value = %.2f\n", result);
    return result;
}
/*
    E' -> + T E'
    E' -> - T E'
    E' -> epsilon
    inh = inherited attribute
*/
double EPrime(double inh)
{
    double tValue;
    double newValue;
    skipSpaces();
    /*E' -> + T E'*/
    if (expression[pos] == '+')
    {
        pos++;
        printf("\nE' found '+' operator\n");
        tValue = T();
        printf("T produced value = %.2f\n", tValue);
        newValue = inh + tValue;
        printf("Inherited value %.2f + T value %.2f = %.2f\n",
            inh, tValue, newValue);
        /*Pass new value as inherited attribute to next E'*/
        printf("New inherited value passed to E' = %.2f\n",
            newValue);
        return EPrime(newValue);
    }

```

```

    }
    /*E' -> - T E'*/
    if (expression[pos] == '-')
    {
        pos++;
        printf("\nE' found '-' operator\n");
        tValue = T();
        printf("T produced value = %.2f\n", tValue);
        newValue = inh - tValue;
        printf("Inherited value %.2f - T value %.2f = %.2f\n",
            inh, tValue, newValue);
        printf("New inherited value passed to E' = %.2f\n",
            newValue);
        return EPrime(newValue);
    }
    /* E' -> epsilon*/
    printf("\nE' -> epsilon\n");
    return inh;
}
/* T -> F T'*/
double T()
{
    double fValue;
    double result;
    printf("\nEntering T (Term)\n");
    fValue = F();
    printf("F produced value = %.2f\n", fValue);
    /*Pass F's value as inherited attribute to T */
    printf("Inherited value passed to T' = %.2f\n", fValue);

```

```

    result = TPrime(fValue);
    printf("T returns value = %.2f\n", result);
    return result;
}

/*
    T' -> * F T'
    T' -> / F T'
    T' -> epsilon
    inh = inherited attribute
*/

double TPrime(double inh)
{
    double fValue;
    double newValue;
    skipSpaces();
    /*T' -> * F T'*/
    if (expression[pos] == '*')
    {
        pos++;
        printf("\nT' found '*' operator\n");
        fValue = F();
        printf("F produced value = %.2f\n", fValue);
        newValue = inh * fValue;
        printf("Inherited value %.2f * F value %.2f = %.2f\n",
            inh, fValue, newValue);
        printf("New inherited value passed to T' = %.2f\n",
            newValue);
        return TPrime(newValue);
    }

```

```

/* T' -> / F T'*/
if (expression[pos] == '/')
{
    pos++;
    printf("\nT' found '/' operator\n");
    fValue = F();
    if (fValue == 0)
    {
        printf("Error: Division by zero!\n");
        return 0;
    }
    printf("F produced value = %.2f\n", fValue);
    newValue = inh / fValue;
    printf("Inherited value %.2f / F value %.2f = %.2f\n",
        inh, fValue, newValue);
    printf("New inherited value passed to T' = %.2f\n",
        newValue);
    return TPrime(newValue);
}

/*T' -> epsilon*/
printf("\nT' -> epsilon\n");
return inh;
}

/*
    F -> id
    F -> ( E )
*/
double F()
{

```



```

double value;
skipSpaces();
/* Parenthesized expression*/
if (expression[pos] == '(')
{
    pos++;
    printf("\nF -> ( E )\n");
    value = E();
    skipSpaces();
    if (expression[pos] == ')')
    {
        pos++;
    }
    else
    {
        printf("Error: Missing closing parenthesis\n");
    }
    return value;
}
/*Variable */
if (expression[pos] == 'a' ||
    expression[pos] == 'b' ||
    expression[pos] == 'c')
{
    char variable = expression[pos];
    pos++;
    value = getValue(variable);
    printf("\nF -> %c\n", variable);
    printf("Value of %c = %.2f\n", variable, value);
}

```

```

        return value;
    }

    printf("Error: Invalid expression\n");
    return 0;
}

/* Main function */
int main()
{
    double result;

    printf("\n");
    printf(" L-ATTRIBUTED DEFINITION EVALUATION\n");
    printf("\n");
    printf("\nGrammar:\n");
    printf("E -> T E\n");
    printf("E' -> + T E' | - T E' | epsilon\n");
    printf("T -> F T\n");
    printf("T' -> * F T' | / F T' | epsilon\n");
    printf("F -> id | ( E )\n");

    /* Input expression */
    printf("\nEnter arithmetic expression: ");
    scanf("%s", expression);

    /* Input variable values */
    printf("\nEnter value of a: ");
    scanf("%lf", &a);

    printf("Enter value of b: ");
    scanf("%lf", &b);

    printf("Enter value of c: ");
    scanf("%lf", &c);

    printf("\n");

```

```

printf(" TOP-DOWN PARSING AND EVALUATION\n");

/* Start parsing from E */

result = E();

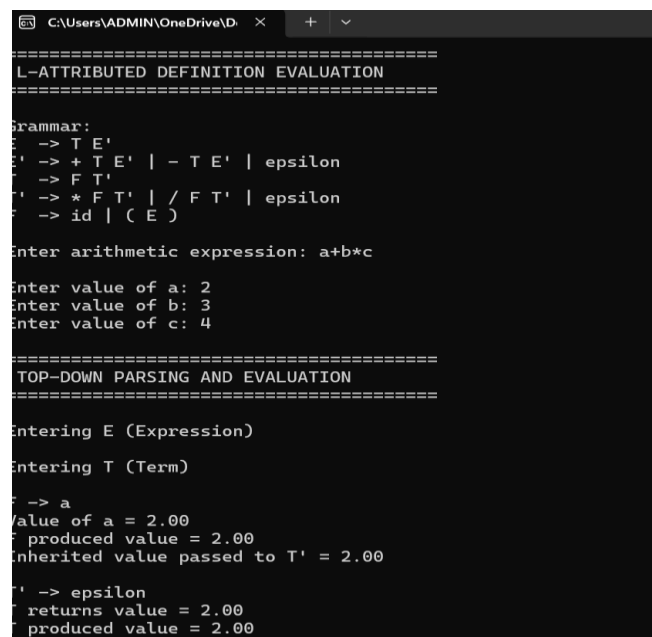
skipSpaces();

if (expression[pos] != '\0')
{
    printf("\nError: Invalid expression!\n");
}
else
{
    printf("\n ");
    printf(" FINAL RESULT = %.2f\n", result);
    printf("");
}

return 0;
}

```

Output:



```

C:\Users\ADMIN\OneDrive\D: x + v
=====
L-ATTRIBUTED DEFINITION EVALUATION
=====

Grammar:
E -> T E'
E' -> + T E' | - T E' | epsilon
E' -> F T'
T' -> * F T' | / F T' | epsilon
T' -> id | ( E )

Enter arithmetic expression: a+b*c

Enter value of a: 2
Enter value of b: 3
Enter value of c: 4

=====
TOP-DOWN PARSING AND EVALUATION
=====

Entering E (Expression)
Entering T (Term)
E -> a
value of a = 2.00
E produced value = 2.00
Inherited value passed to T' = 2.00

T' -> epsilon
T' returns value = 2.00
T' produced value = 2.00

```

```
C:\Users\ADMIN\OneDrive\D... x + ~
E' found '+' operator
Entering T (Term)
F -> b
Value of b = 3.00
F produced value = 3.00
Inherited value passed to T' = 3.00
T' found '*' operator
F -> c
Value of c = 4.00
F produced value = 4.00
Inherited value 3.00 * F value 4.00 = 12.00
New inherited value passed to T' = 12.00
T' -> epsilon
T returns value = 12.00
T produced value = 12.00
Inherited value 2.00 + T value 12.00 = 14.00
New inherited value passed to E' = 14.00
E' -> epsilon
E returns value = 14.00

=====
FINAL RESULT = 14.00
=====

-----
Process exited after 13.63 seconds with return value 0
Press any key to continue . . . |
```

Q4.

A compiler's semantic analyzer must check type compatibility between expressions. The system should:

- (i) Accept two type expressions such as int, float, int*, char*
- (ii) Compare the types for equivalence
- (iii) Handle both basic and pointer types
- (iv) Display whether the types are equivalent or not

Write a C program to perform type equivalence checking and explain how compilers use this mechanism during semantic analysis.

Program:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
/*
```

Type Equivalence Checker

Basic Types:

int

float

char

Pointer Types:

int*

float*

char*

*/

/* Function to check whether a type is valid */

int isValidType(char type[])

{

if (strcmp(type, "int") == 0)

return 1;

if (strcmp(type, "float") == 0)

return 1;

if (strcmp(type, "char") == 0)

return 1;

if (strcmp(type, "int*") == 0)

return 1;

if (strcmp(type, "float*") == 0)

return 1;

if (strcmp(type, "char*") == 0)

return 1;

return 0;

}

/* Function to check whether a type is a pointer */

int isPointer(char type[])

{

int length = strlen(type);

if (type[length - 1] == '*')

return 1;

```

    return 0;
}

/* Function to get the base type of a pointer */
void getBaseType(char type[], char baseType[])
{
    int length = strlen(type);
    strncpy(baseType, type, length - 1);
    baseType[length - 1] = '\0';
}

/* Function to check type equivalence */
int areEquivalent(char type1[], char type2[])
{
    char base1[20];
    char base2[20];

    /* Case 1: Both are basic types */
    if (!isPointer(type1) && !isPointer(type2))
    {
        if (strcmp(type1, type2) == 0)
            return 1;
        else
            return 0;
    }

    /* Case 2: One is pointer and the other is basic */
    if (isPointer(type1) != isPointer(type2))
    {
        return 0;
    }

    /* Case 3: Both are pointer types */
    getBaseType(type1, base1);

```

```

    getBaseType(type2, base2);
    if (strcmp(base1, base2) == 0)
        return 1;
    return 0;
}

/* Main function */
int main()
{
    char type1[20];
    char type2[20];
    printf("\n");
    printf("    TYPE EQUIVALENCE CHECKER\n");
    printf("\n");
    printf("\nSupported Types:\n");
    printf("Basic : int, float, char\n");
    printf("Pointer: int*, float*, char*\n");
    /* Read first type */
    printf("\nEnter first type: ");
    scanf("%s", type1);
    /* Read second type */
    printf("Enter second type: ");
    scanf("%s", type2);
    /* Validate types */
    if (!isValidType(type1))
    {
        printf("\nInvalid first type!\n");
        return 0;
    }
    if (!isValidType(type2))

```

```

{
    printf("\nInvalid second type!\n");
    return 0;
}

/* Display comparison */
printf("\n ");
printf("Type 1: %s\n", type1);
printf("Type 2: %s\n", type2);
printf("\n");

/* Check equivalence */
if (areEquivalent(type1, type2))
{
    printf("Result: Types are EQUIVALENT.\n");
}
else
{
    printf("Result: Types are NOT EQUIVALENT.\n");
}

printf("\n");

return 0;
}

```

Output:

```

C:\Users\ADMIN\Documents\
=====
TYPE EQUIVALENCE CHECKER
=====

Supported Types:
Basic : int, float, char
Pointer: int*, float*, char*

Enter first type: int
Enter second type: float

-----
Type 1: int
Type 2: float
-----
Result: Types are NOT EQUIVALENT.
=====

-----
Process exited after 14.03 seconds with return value 0
Press any key to continue . . . |

```


Q5.

You are designing a semantic analyzer for a compiler that detects type errors in arithmetic expressions. The system must:

- (i) Accept operands along with their types
- (ii) Validate operations such as +, -, , /
- (iii) Detect invalid combinations such as int + char
- (iv) Report whether the expression is valid or contains a type error

Write a C program to perform type checking and explain how semantic analysis ensures correctness in program execution.

Program:

```
print("    TYPE CHECKING SYSTEM")
print("    SEMANTIC ANALYZER")
# Get first operand and type
operand1 = input("\nEnter first operand: ")
type1 = input("Enter type of first operand (int/float/char): ").lower()
# Get operator
operator = input("Enter operator (+, -, *, /): ")
# Get second operand and type
operand2 = input("Enter second operand: ")
type2 = input("Enter type of second operand (int/float/char): ").lower()
# Check whether entered types are valid
valid_types = ["int", "float", "char"]
if type1 not in valid_types:
    print("\nError: Invalid first type!")
    exit()
if type2 not in valid_types:
    print("\nError: Invalid second type!")
    exit()
# Check whether operator is valid
```

```

valid_operators = ["+", "-", "*", "/"]
if operator not in valid_operators:
    print("\nError: Invalid operator!")
    exit()

print("\n")
print("SEMANTIC ANALYSIS")
print("First operand :", operand1)
print("First type   :", type1)
print("Operator     :", operator)
print("Second operand :", operand2)
print("Second type   :", type2)
print("\nChecking expression:")
print(type1, operator, type2)

valid = False

# int with int
if type1 == "int" and type2 == "int":
    valid = True

# float with float
elif type1 == "float" and type2 == "float":
    valid = True

# int with float
elif type1 == "int" and type2 == "float":
    valid = True

# float with int
elif type1 == "float" and type2 == "int":
    valid = True

# All other combinations are invalid
else:
    valid = False

```

```

print("\n")

if valid:

    print("Result: VALID EXPRESSION")

    print("No type error detected.")

    # Determine result type

    if type1 == "float" or type2 == "float":

        result_type = "float"

    else:

        result_type = "int"

    print("Result type:", result_type)

else:

    print("Result: TYPE ERROR")

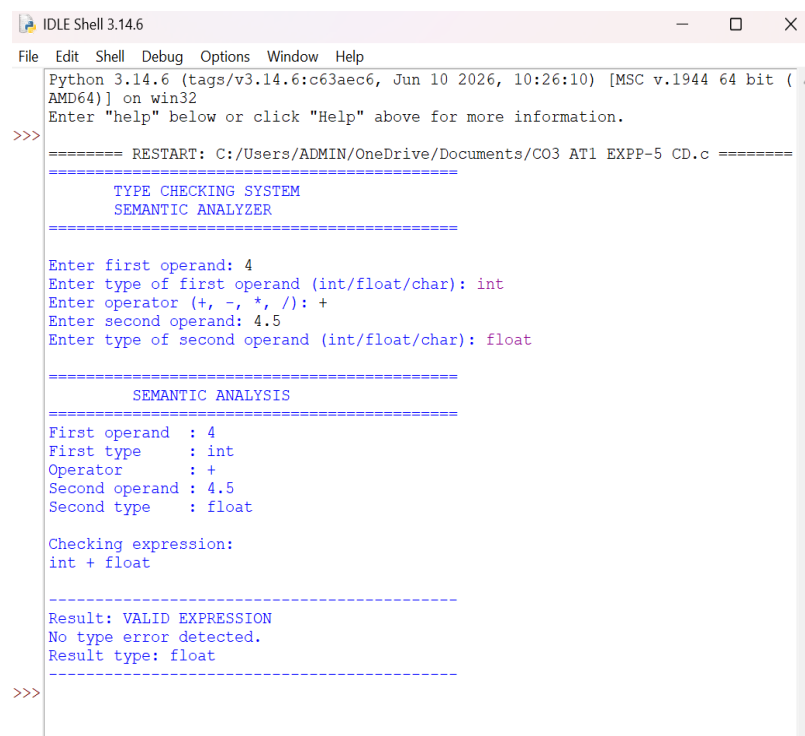
    print("Invalid type combination:")

    print(type1, operator, type2)

print("")

```

Output:



```

IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ADMIN/OneDrive/Documents/CO3 AT1 EXPP-5 CD.c =====
=====
      TYPE CHECKING SYSTEM
      SEMANTIC ANALYZER
=====

Enter first operand: 4
Enter type of first operand (int/float/char): int
Enter operator (+, -, *, /): +
Enter second operand: 4.5
Enter type of second operand (int/float/char): float

=====
      SEMANTIC ANALYSIS
=====
First operand : 4
First type    : int
Operator      : +
Second operand : 4.5
Second type   : float

Checking expression:
int + float

-----
Result: VALID EXPRESSION
No type error detected.
Result type: float
-----
>>>

```

